

Desafio Avançado

Processos de Desenvolvimento de Software · Avaliação

Desafio 1 — Cascata × Scrum no mesmo requisito

Requisito escolhido: “O sistema deve manter uma fila de espera de consultas: quando não há horário livre, o paciente entra na fila e é chamado automaticamente quando um horário é liberado.” (AgendaClínica)

Plano em Cascata: Requisitos: levantar e aprovar TODAS as regras da fila antes de começar (ordem por chegada? prioridade para idosos? tempo máximo na fila? notificação por SMS ou app?). Projeto: modelar a estrutura de dados da fila, as telas (entrar, ver posição, ser chamado) e a regra “liberou horário → chama o próximo”. Implementação: programar conforme o projeto. Testes: entrada, saída, ordem, notificação, casos de erro. Manutenção: ajustes após a entrega. Entrega única, ao final.

Plano em Scrum: Sprint 1: fila simples por ordem de chegada + tela “você está na posição X”; demo ao PO. Sprint 2: chamada automática ao liberar horário + notificação; demo. Sprint 3: regra de prioridade (idosos/urgência) e tempo máximo na fila, conforme o feedback das sprints anteriores; demo. Entrega incremental a cada sprint.

Comparação — mudança de requisito no meio do caminho (ex.: incluir gestantes na prioridade): Cascata: com a implementação já em andamento, é preciso reabrir os documentos de requisitos e de projeto, alterar código já testado e retestar tudo; custo e prazo sobem e a mudança “compete” com um cronograma fechado. Scrum: a nova regra vira um item do backlog e entra na próxima sprint; o que já foi entregue continua funcionando; o impacto fica contido em uma sprint. Conclusão: como as regras de fila evoluem com o uso real, o Scrum absorve mudança com muito menos retrabalho; o Cascata só compensaria se todas as regras estivessem estáveis desde o início.

Desafio 2 — Kanban com dados reais

Quadro Kanban (draw.io): Colunas: To Do, Doing, Review, Done. Limites de WIP: Doing = 3, Review = 2. Dez itens de trabalho (T1 a T10) movidos ao longo de uma semana simulada (segunda a sexta).

Evolução dia a dia (nº de itens por coluna; total sempre 10): Seg: To Do 8 · Doing 2 · Review 0 · Done 0. Ter: 6 · 3 · 1 · 0. Qua: 4 · 3 · 1 · 2. Qui: 2 · 3 · 1 · 4. Sex: 0 · 2 · 2 · 6.

Cumulative Flow Diagram (esboço manual): Eixo X = dias (Seg a Sex); eixo Y = nº acumulado de itens; faixas empilhadas de baixo para cima: Done, Review, Doing, To Do. Leitura: a faixa Done (embaixo) cresce de forma constante (≈ 2 itens/dia a partir de quarta) = fluxo saudável; a faixa To Do (em cima) encolhe = backlog sendo consumido; as faixas Doing e Review

ficam finas e estáveis = não há gargalo. Se a faixa Review fosse ficando cada vez mais larga, seria sinal de gargalo na revisão.

Desafio 3 — Pipeline de CI mais completo (flake8 / pylint)

Nova etapa adicionada: Um passo “Lint” que roda antes dos testes: instala o flake8 e executa “flake8.” (ou “pylint src/”). Se encontrar problemas, o passo falha e os testes nem chegam a rodar.

Trecho do workflow:

```
- name: Lint (flake8)
  run: |
    pip install flake8
    flake8 . --max-line-length=100

- name: Testes
  run: pytest
```

O que essa etapa verifica: Estilo e padrões do código Python (PEP 8): indentação, linhas muito longas, imports não usados, variáveis declaradas e nunca usadas, nomes fora do padrão, espaçamento inconsistente. O pylint ainda aponta “code smells” (funções muito longas, complexidade alta) e dá uma nota ao código.

Por que é útil além dos testes funcionais: Os testes dizem se o código FAZ a coisa certa; o lint diz se o código está LEGÍVEL e consistente. Isso previne bugs futuros (um import não usado pode esconder um erro; uma variável não usada pode indicar lógica incompleta), acelera a revisão de código e mantém o projeto uniforme quando várias pessoas mexem nele. No pipeline, ninguém consegue dar merge de código fora do padrão.

Desafio 4 — Auditoria de qualidade de requisitos

Checklist aplicado (Cap. 6): Para cada requisito: (1) está claro, sem ambiguidade? (2) é testável/verificável? (3) é atômico (um requisito só)? (4) não mistura função com qualidade (F × NF)? (5) tem critério de aceitação mensurável?

Amostra da auditoria: RF-01 “Marcar consulta escolhendo médico, data e horário” → passou. RF-02 “O sistema deve ser rápido” → reprovou nos critérios 1, 2 e 5; ajuste: “carregar a agenda em até 3 s”. RF-03 “Enviar lembrete por SMS 24h antes e permitir remarcar” → reprovou no critério 3 (dois requisitos juntos); ajuste: separar em RF-03a e RF-03b. RF-04 “Manter fila de espera e ser seguro” → reprovou no critério 4 (mistura F e NF); ajuste: RF-04 (fila) + RNF-01 (segurança com critério). RNF-01 “Login guardado com criptografia” → passou. RF-05 “Cancelar até 2h antes sem multa” → passou. RF-06 “Tela bonita e intuitiva” → reprovou nos critérios 1, 2 e 5; ajuste: “um novo usuário conclui a marcação em até 3 telas”. RF-07 “Relatório mensal por médico” → passou.

Relatório de auditoria (resumo): De 8 requisitos: 4 passaram sem ressalvas, 4 precisaram de ajuste. Problemas mais comuns: (a) falta de critério de aceitação mensurável

(adjetivos como “rápido”, “bonito”); (b) requisitos não atômicos (dois numa frase só); (c) mistura de funcional com não funcional. Recomendação: padronizar a escrita de requisito (ID, tipo, descrição testável, critério de aceitação, prioridade) e revisar todo requisito novo contra o checklist antes de aprová-lo.

Exemplo preenchido pela equipe pedagógica — referência de correção. Situação fictícia produzida pela SEDUC-SP.